

Architecture Narrative

CMU Research Matchmaker | 95-731 AI, Cloud, and DevOps | Group 8 Team Milestone 1
Authors: Jason Liu, Kaitlin Moore, Jorge Urias, Mel Wong

What we built

The system is a research mentor matchmaker for CMU students. A student asks a question in plain language, and the service returns a ranked list of CMU researchers, each with a one-line rationale grounded in that researcher's own publications and cited against specific works. Publication data comes from the OpenAlex API, rolled into per-researcher profiles, embedded, and indexed for similarity search.

The service is a FastAPI application in Docker, and the container is the unit of truth. The same image runs in every teammate's devcontainer and on Fargate, so local behavior and deployed behavior cannot drift apart. The image is built in two stages. Ingestion and embedding run during the build, which means the FAISS index and profile data ship inside the image and a fresh clone serves end to end with a single build and run command, no separate data-preparation step.

The code is organized as one package per module (adapters, ingest, profiles, match, agent, api), and each boundary is defined by the data contracts in `docs/contracts.md` (RawRecord, ResearcherProfile, MatchResult, RankedAnswer). Those contracts are the seam that decouples the retrieval half of the system from the serving half. Matching is deterministic, with no language model anywhere in the ranking path, and the rationale attached to each result is a template built from the matched profile's own topics and works. Every record carries provenance (source, fetch time, trust tier) stamped once at ingestion.

Cloud services and why

We deploy the image to ECR and run it on ECS Fargate under the pre-provisioned LabRole, inside the AWS Academy Learner Lab. Fargate was chosen because it is confirmed available in the lab, while App Runner and Bedrock are documented absent from the lab's service list, and because a single Fargate task matches the shape of the artifact (one self-contained container). Deployment is fully scripted (`deploy/`) and reproducible in any team member's lab session, and the service is ephemeral by design because lab sessions are. Development occurred in devcontainers with the same container deployed and verified in the Learner Lab sandbox.

Key decisions

Decision	Reasoning
Corpus bounded to works 2021+, 3 departments (~100 authors, 20 works/author)	Keeps build-time embedding cheap on every machine and makes the known-item evaluation constructible; a broad corpus adds compute cost without strengthening the prototype. Machine Learning (MLD), Language Technologies Institute (LTI), and Human-Computer Interaction (HCII) were chosen as the departments to give good publication density and related, but distinct topical coverage.
all-MiniLM-L6-v2 rather than a scholarly model (SPECTER2)	At this corpus density, researchers are far apart in topic space, so the smaller model clears the evaluation bar. The swap to upgrade to SPECTER2 is one constructor argument plus an index rebuild.
Match evidence populated heuristically (topic intersection + recency)	Mean-of-profile vectors cannot attribute a match to specific works. Stating the heuristic openly keeps the contract honest, and exact attribution arrives with per-paper multi-vector matching.
Index baked into the image rather than external storage	Consistent with container-as-truth, there is no S3 or database dependency, and a rebuild is just a container build
Uniform error contract	Every failure mode returns the same <code>{"error", "message"}</code> shape, so clients never branch on how the backend failed.

Evaluation design

The designed evaluation is a known-item retrieval test. We select ten identifiable CMU faculty within the corpus departments (MLD, LTI, HCII) and write one query each that should surface that person. The pass criterion is that the expected researcher ranks in the top five for their query. The results table in `docs/evaluation.md` is the template for this test, awaiting execution against the full pipeline.

Upgrade paths

Every simplification in this version was chosen with a documented exit. The system's contracts (RawRecord, ResearcherProfile, MatchResult, RankedAnswer) were designed so that each deferred capability attaches as a new implementation behind an existing interface, and the response shape a client sees never changes. The table below summarizes the paths, and the paragraphs that follow give the reasoning for the three most consequential ones.

Deferred capability	What ships instead (v1)	Where the upgrade attaches	Cost of upgrading
Scholarly embedding model (SPECTER2 / SciNCL)	all-MiniLM-L6-v2, general-purpose	Embedding module, model constructor	Larger image, adapter-based loading for SPECTER2, full index rebuild (one docker build)
Per-paper multi-vector matching with recency weighting	Mean-of-profile vector, cosine top-N	Internal to embedding, vector store, and match engine	Index restructure (one row per paper), scoring change, no contract change
Exact match attribution	Heuristic evidence (topic intersection, recent works)	Match engine output logic	Arrives as a byproduct of multi-vector matching; the fields already exist in MatchResult
LLM-generated rationale	Template rationale from matched profile data	AgentRuntime interface, second implementation	Model API choice, funded key, per-query cost cap, prompt-injection posture if web-enabled
Live web enrichment	<code>search_effort</code> off, <code>web_enrichment</code> empty	Web search adapter, one interface implementation	Provider key, cost and latency budget, citation and trust-tier handling already specified
Additional data sources (Scholars@CMU, directory, KiltHub)	OpenAlex only	New source adapters producing RawRecord	Per-source access work; Elements API requires institutional coordination
Broader corpus	2021+, 3 departments, ~100 authors, 20 works per author	Ingestion configuration	Longer build-time embedding run; motivates the storage upgrade below
External index storage (S3 or pgvector)	Index baked into the container image at build	Build pipeline and API startup path	Adds a startup dependency and AWS footprint; warranted only when corpus growth makes image baking slow

Embedding model. The v1 model is general-purpose rather than scholarly-trained. The scholarly alternatives (SPECTER2, trained on citation structure, or SciNCL) rank fine-grained topical distinctions better, and that advantage grows with corpus density. At roughly 100 authors across 3 departments, researchers are spread far enough apart in topic space that the smaller model clears the evaluation bar, while keeping the image lean and the build fast for every teammate. The swap is isolated to the embedding module's model constructor, with the one real cost being an index rebuild, since vectors from different models are not comparable. In this architecture a rebuild is a container build, so the upgrade is cheap by design.

Matching depth. The v1 match engine scores a query against one averaged vector per researcher. Averaging blurs multi-topic researchers, and the planned upgrade (one vector per paper, max-similarity scoring, recency weighting) addresses exactly that. It also converts the match evidence fields from heuristic to exact attribution, since per-paper scoring knows which works drove a match. The upgrade is entirely internal to the

retrieval half; MatchResult's shape was written to accommodate it from the start, which is why the evidence fields exist now and are populated heuristically rather than omitted.

Rationale generation. The v1 rationale is a template built from the matched profile's own topics and works, which keeps the answer grounded and the system free of model-API dependencies. The AgentRuntime interface exists so that an LLM-backed implementation, with or without a web-search tool, can be added as a second implementation rather than a redesign. Activating it requires decisions this version deliberately avoided, a model provider, a funded key, a per-query cost cap, and, if web search is enabled, citation handling and prompt-injection guardrails that were specified during scoping and would need to be implemented alongside it. Deferring those decisions is what kept the build unblocked.

The common property across all eight paths is that the client-facing response shape never changes. Fields like `web_enrichment`, `search_effort_used`, and the match evidence pair are present in every v1 response, carrying empty or heuristic values, so each upgrade fills in existing structure rather than breaking consumers. That choice cost little in v1 and is what makes the boundary between shipped and deferred a designed seam rather than an accident of time pressure.